

Class Exercise:

Exploring Search Algorithms

Objective:

- Understand and implement common search algorithms.
- Compare the time complexity and performance of different search algorithms.

Problem: You are given a sorted array of integers and an unsorted array of integers. Your task is to implement three search algorithms and compare their performance.

Sample Data:

- Sorted Array: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
- Unsorted Array: [13, 3, 19, 1, 7, 11, 9, 5, 15, 17]
- Target: 9

Class Exercise:

Exploring Search Algorithms

Task 1: Linear Search

1. Implement **Linear Search** to find an element in the unsorted array.

2.Code:

- **Function Signature:** `linear_search(arr: List[int], target: int) -> int`
- The function should return the index of the target element if found, otherwise return -1.

Class Exercise:

Exploring Search Algorithms

Task 2: Binary Search

1. Implement **Binary Search** to find an element in the sorted array.

2. Code:

- **Function Signature:** `binary_search(arr: List[int], target: int) -> int`
- The function should return the index of the target element if found, otherwise return -1.
- Note: Binary Search only works on sorted arrays, so you must sort the array first.

Class Exercise:

Exploring Search Algorithms

Task 3: Comparison

1. Given an array, compare the time taken to search for an element using Linear Search and Binary Search.

2. Code:

- Use the **time module in Python** to measure the time taken for each search method.
- Test with both small and large arrays.

Class Exercise:

Exploring Search Algorithms

Task 4: Discussion

1. Analyze the time complexities of each search algorithm:

1. Linear Search: $O(n)$

2. Binary Search: $O(\log n)$

2. In what situations would you prefer one algorithm over the other? Provide examples.